

Seccomp-BPF sur Android

Filtrage des appels système

Groupe 10 : Labo 03 - SOS

Kevin Auberson, Dani Tiago Faria dos Santos, Jonhatan Thiébaud

Agenda

1. **Introduction** - C'est quoi Seccomp-BPF ?
2. **Fonctionnement** - Comment ça filtre les syscalls
3. **Syscalls** - Autorisés vs Bloqués
4. **Seccomp-BPF** - Sur Android
5. **Impact du blocage** - Exemples
6. **Demo live** - Application de test + strace
7. **Conclusion**

1. C'est quoi Seccomp-BPF ?

Secure Computing with Berkeley Packet Filter

Seccomp-BPF = mécanisme kernel Linux qui **filtre les appels système**

- Chaque app Android passe par ce filtre
- Configuré par **Bionic** (la libc Android)
- Si syscall interdit → **SIGKILL** (process tué)

Pourquoi c'est important ?

Protection contre l'élévation de privileges

Une app malveillante pourrait essayer:

- `setuid(0)` → devenir root
- `reboot()` → redémarrer le téléphone
- `init_module()` → charger du code kernel

Seccomp-BPF bloque tout ça au niveau kernel

2. Fonctionnement

Flux d'un appel système



Le filtre BPF

BPF = mini machine virtuelle dans le kernel

```
// Pseudo-code du filtre Seccomp Android
if (syscall == SYS_setuid) {
    return SECCOMP_RET_KILL;    // Tue le process!
}
if (syscall == SYS_read) {
    return SECCOMP_RET_ALLOW;   // OK, autorise
}
```

Vérifier Seccomp sur Android

```
# Actions disponibles
adb shell cat /proc/sys/kernel/secomp/actions_avail
kill_process kill_thread trap errno trace log allow

# Status d'un processus (2 = filter BPF actif)
adb shell cat /proc/<PID>/status | grep Seccomp
Seccomp:      2
```

0 = off | **1** = strict | **2** = **filter (BPF)**

3. Syscalls AUTORISES

Catégorie	Syscalls	Description
Fichiers	openat, read, write, close	Lecture/écriture
Réseau	socket, connect, sendto, recvfrom	Communication
Mémoire	mmap, mprotect, brk	Allocation
Process	getpid, getuid, clone	Info process
Temps	clock_gettime, nanosleep	Timing

3. Syscalls BLOQUES

Catégorie	Syscalls	Raison
Privileges	setuid, setgid, setgroups	Elevation privileges
Systeme	reboot, sethostname	Modification système
Temps	settimeofday, clock_settime	Manipulation horloge
Kernel	init_module, delete_module	Chargement modules
Root	chroot, swapon, swapoff	Accès root

→ SIGKILL si appelé

4. Impact du blocage - Exemples

- Comparaison avant/après Seccomp
- Scénarios d'attaque bloqués
- Processus de détection et terminaison

5. Limitations de Seccomp-BPF

- Ne protège pas contre toutes les attaques
- Dépend de la configuration Bionic
- Performance overhead minimal
- Complémentaire à SELinux

6. Sandboxing Android

Chaque app = un utilisateur Linux

```
App Instagram → UID 10045 → /data/user/0/com.instagram/  
App WhatsApp → UID 10078 → /data/user/0/com.whatsapp/  
Notre app     → UID 10192 → /data/user/0/com.demo.ebpf/
```

- UIDs commencent à **10000** (AID_APP_START)
- Chaque app est **isolée** des autres
- Pas d'accès aux fichiers des autres apps

Sandboxing - Couches de protection

Couche	Mecanisme	Rôle
1. UID	Utilisateur Linux unique	Isolation fichiers
2. SELinux	Mandatory Access Control	Politique de sécurité
3. Seccomp-BPF	Filtrage syscalls	Bloquer syscalls dangereux

Seccomp-BPF = dernière ligne de défense

Si l'app passe UID + SELinux, Seccomp bloque les syscalls interdits

7. Demo Live

Notre application de test

8 boutons pour déclencher des syscalls:

Réseau	Fichiers	Systeme
HTTP GET	Créer	getpid()
DNS Lookup	Lire	clock_gettime()
	Supprimer	
	Lister	setuid(0)

Demo - Monitoring avec strace

strace = outil pour tracer les syscalls

```
# Attacher strace a notre app  
adb shell strace -p <PID> -f
```

On voit en live chaque syscall exécuté par l'app

Demo - Monitoring avec strace

On ignore les parties suivantes dans le print :

- attached/detached : messages strace d'attachement au process
- unfinished/resumed : syscalls interrompus puis repris
- EAGAIN/EINTR/ERESTART : erreurs temporaires(retry)
- ENOENT/EACCES : fichier non trouvé / accès refusé

Quelques résidus de bruit restent sinon on ne vois presque plus rien.

Demo - HTTP GET

```
[17:25:01] [NET] socket() → création socket TCP/IP  
[17:25:01] [NET] connect() → 93.184.216.34:80  
[17:25:01] [NET] sendto() → 245 octets envoyés  
[17:25:01] [NET] recvfrom() → 1256 octets reçus
```

Syscalls réseau autorisés → requête HTTP réussie

Demo - Créer fichier

```
[17:25:05] [FILE] openat() → test_1705123456.txt  
[17:25:05] [FILE] write() → 48 octets écrits  
[17:25:05] [FILE] close() → fichier fermé
```

Syscalls fichiers autorisés → fichier créé

Demo - getpid() / getuid()

```
[17:25:10] [SYS] getpid() → 12345  
[17:25:10] [SYS] getuid() → 10156
```

Syscalls système autorisés → info process obtenue

Demo - `setuid(0)`

Tentative d'élévation de privileges

```
// Dans notre app Android  
Os.setuid(0); // Essayer de devenir root
```

Demo - `setuid(0)` BLOQUE

```
[17:25:15] [!!!!] Execution setuid(0)...  
Process 12345 terminated by SIGKILL
```

L'application CRASH instantanément

Seccomp-BPF a intercepté le syscall interdit
→ Le kernel a tué le processus

5. Conclusion

Seccomp-BPF = dernière ligne de défense

Avant	Après
App tente <code>setuid(0)</code>	Seccomp intercepte
Syscall part vers kernel	Filtre BPF analyse
App devient root	SIGKILL → app tuée

Points clés

1. Filtrage au niveau kernel

Impossible à contourner depuis l'application

2. Liste blanche de syscalls

Seuls les syscalls nécessaires sont autorisés

3. Sanction immédiate

Syscall interdit = **SIGKILL** (pas d'exception)

Références

- **Seccomp-BPF Android (Bionic)**
`android.googlesource.com/platform/bionic`
- **Linux Seccomp man page**
`man7.org/linux/man-pages/man2/seccomp.2.html`
- **Android Security Overview**
`source.android.com/security`

Questions ?

Merci de votre attention